

# Sistema de intersecciones y controles de tráfico – Descripción limpia

## 1. Función general del sistema

Este conjunto de blueprints controla el comportamiento de las intersecciones, los semáforos, las señales de STOP, las señales de YIELD y las zonas de cruce que deben estar libres antes de que un coche pueda continuar.

El objetivo principal del sistema es coordinar la circulación de los coches inteligentes en puntos conflictivos del mapa. Para ello, cada control de tráfico informa al `BP_AiController` del coche cuándo debe detenerse, dónde está la línea de parada, qué tipo de control está afectando al vehículo y cuándo puede volver a circular.

El sistema también evalúa si un coche cumple correctamente las normas. Si el vehículo atraviesa un semáforo en rojo, no realiza correctamente un STOP o no respeta un YIELD, se le aplica una penalización al fitness y se llama a `KillCar` con la razón de muerte correspondiente.

Además, algunos umbrales no son completamente fijos. Tanto el semáforo como el STOP almacenan muestras legales de velocidad o tiempo de parada para aprender valores de referencia. Cuando todavía no existen suficientes muestras, se usan valores iniciales de seguridad.

Los blueprints principales de este sistema son:

- `BP_IntersectionController`, que alterna las fases de los semáforos.
- `BP_TrafficLight`, que representa cada semáforo individual.
- `BP_Sign_Stop`, que controla la lógica de una señal de STOP.
- `BP_Sign_Yield`, que controla la lógica de una señal de ceda el paso.
- `BP_CrossingZone`, que define una zona de cruce que debe estar libre.
- `S_StopSample`, que almacena muestras legales de parada.
- `BI_TrafficControl`, que define una interfaz básica para controles de tráfico.

---

## BP\_IntersectionController

### 2. Función general

El `BP_IntersectionController` es el blueprint que coordina el ciclo completo de los semáforos de una intersección.

Su función es alternar entre dos grupos de semáforos:

- `Lights_NS`, que representa los semáforos del eje norte-sur.
- `Lights_EW`, que representa los semáforos del eje este-oeste.

El controlador garantiza que solo una dirección tenga luz verde a la vez y añade fases intermedias de ámbar y todo rojo para evitar conflictos entre vehículos que puedan estar cruzando la intersección.

---

### 3. Variables principales

`Lights_NS` es un array editable que contiene los semáforos correspondientes a la dirección norte-sur.

`Lights_EW` es un array editable que contiene los semáforos correspondientes a la dirección este-oeste.

`Time_Green` define cuánto dura la fase verde. Su valor inicial es `20.0` segundos.

`Time_Amber` define cuánto dura la fase ámbar. Su valor inicial es `3.0` segundos.

`Time_AllRed` define cuánto dura la fase en la que todos los semáforos implicados están en rojo. Su valor inicial es `3.0` segundos.

---

### 4. Flujo inicial: Event BeginPlay

Cuando empieza la simulación, el controlador recorre todos los semáforos de `Lights_EW` y los pone en rojo llamando a `SetLightState` con el estado `red`.

Después llama al evento `Phase_NS_Green`, iniciando el ciclo con la dirección norte-sur en verde.

---

### 5. Ciclo de fases del semáforo

El ciclo de la intersección se compone de seis fases que se llaman unas a otras de forma continua.

#### Phase\_NS\_Green

En esta fase, todos los semáforos de `Lights_NS` se ponen en verde.

Después el sistema espera `Time_Green` segundos y llama a `Phase_NS_Amber`.

#### Phase\_NS\_Amber

En esta fase, todos los semáforos de `Lights_NS` pasan a ámbar.

Después espera `Time_Amber` segundos y llama a `Phase_AllRed_1`.

### **Phase\_AllRed\_1**

En esta fase, todos los semáforos de `Lights_NS` se ponen en rojo.

Después espera `Time_AllRed` segundos y llama a `Phase_EW_Green`.

Esta fase de todo rojo funciona como margen de seguridad antes de dar paso al tráfico del otro eje.

### **Phase\_EW\_Green**

En esta fase, todos los semáforos de `Lights_EW` se ponen en verde.

Después espera `Time_Green` segundos y llama a `Phase_EW_Amber`.

### **Phase\_EW\_Amber**

En esta fase, todos los semáforos de `Lights_EW` pasan a ámbar.

Después espera `Time_Amber` segundos y llama a `Phase_AllRed_2`.

### **Phase\_AllRed\_2**

En esta fase, todos los semáforos de `Lights_EW` se ponen en rojo.

Después espera `Time_AllRed` segundos y vuelve a llamar a `Phase_NS_Green`.

De esta forma, el ciclo se repite indefinidamente durante la simulación.

---

## **BI\_TrafficControl**

### **6. Función general**

`BI_TrafficControl` es una interfaz relacionada con los controles de tráfico.

Define dos funciones principales:

- `TrafficStop`
- `TrafficGo`

Estas funciones representan de forma abstracta las órdenes básicas que puede emitir un control de tráfico: obligar al coche a detenerse o permitirle continuar.

Aunque en este archivo no se detalla su implementación concreta, la interfaz sirve como estructura común para sistemas como semáforos, STOP o YIELD.

---

## BP\_TrafficLight

### 7. Función general

El `BP_TrafficLight` representa un semáforo individual.

Este blueprint se encarga de mostrar visualmente el color actual del semáforo, informar a los coches que entran en su zona de influencia, detectar posibles infracciones y aplicar penalizaciones si un coche cruza cuando no debe.

Cada semáforo tiene una `SensorZone`, que cubre la zona donde el coche debe obedecer la señal, y una flecha `StopLinePosition`, situada justo debajo del semáforo, que indica el punto exacto donde el coche debería detenerse.

---

### 8. Componentes principales

`StopLinePosition` es una flecha situada en la línea de parada del semáforo. Su posición se usa como objetivo de detención para el vehículo.

`SensorZone` es una caja de colisión que cubre toda la zona donde el coche debe reaccionar al semáforo.

---

### 9. Variables principales

`DMI_Roja`, `DMI_Verde` y `DMI_Amarilla` son instancias dinámicas de material. Se usan para modificar el brillo de cada luz del semáforo según el estado actual.

`bIsRedLight` indica si el semáforo está en rojo.

`CurrentState` almacena el estado actual del semáforo usando el enum `E_TrafficLightState`.

`ColorState` representa el estado del semáforo como número:

- `0.0` para rojo.
- `0.5` para ámbar.
- `1.0` para verde.

Este valor se pasa al `BP_AiController` para que el coche pueda interpretarlo fácilmente.

`ViolationPenaltyMultiplier` multiplica la penalización aplicada cuando un coche comete una infracción de semáforo. Su valor es `3.0`.

`LegalTrafficLightsSpeeds` almacena velocidades consideradas legales al salir de la zona del semáforo.

`MinSamplesForLearnedThreshold` define el número mínimo de muestras necesarias para usar un umbral aprendido. Su valor es `10`.

`MaxStoredSamples` limita cuántas muestras legales se conservan. Su valor es `100`.

`LearnedThreshold` guarda el umbral de velocidad aprendido para decidir si un coche se considera detenido o no.

---

## 10. Entrada en la zona del semáforo

Cuando un coche entra en `SensorZone`, se llama a `UpdateTrafficZone`.

Esta función revisa todos los coches que están solapándose con la zona del semáforo y actualiza su controlador.

Para cada coche detectado:

1. Se obtiene su `BP_AiController`.
2. Se asigna `StopTargetLocation` usando la posición mundial de `StopLinePosition`.
3. Se pone `IsTrafficLightActive` en `true`.
4. Se copia `ColorState` en `TrafficLightState`.
5. Se calcula `SensorEntryDistance` como la distancia entre la línea de parada y el coche.

Después, si el semáforo está en rojo o ámbar y el coche todavía no tenía registrada una velocidad de aproximación, se hace lo siguiente:

1. Se establece `TrafficControlType` en `0`, indicando que el control activo es un semáforo.
2. Se guarda `ApproachSpeed` como la velocidad frontal actual del coche.

Si el semáforo deja de estar en estado de parada y el coche tenía una velocidad de aproximación registrada, esa velocidad se reinicia a `0.0`.

---

## 11. Salida de la zona del semáforo

Cuando un coche sale de `SensorZone`, se comprueba si ha respetado correctamente el semáforo.

Primero se obtiene el `BP_VehicleAI2` y su `BP_AiController`. Después se calcula el umbral de velocidad aprendido llamando a `GetLearnedTrafficLightSpeedThreshold`.

Luego se comprueba la infracción:

```
TrafficLightState == 0.0 AND ForwardSpeed > LearnedThreshold
```

Esto significa que, si el semáforo estaba en rojo y el coche sale de la zona con una velocidad superior al umbral permitido, se considera que se ha saltado el semáforo.

Si la condición es verdadera, se llama a `ApplyTrafficLightDeathPenalty`.

Si la condición es falsa, la salida se considera legal. En ese caso:

1. Se añade una muestra legal con la velocidad absoluta del coche.
2. Se reinicia `ApproachSpeed` a `0.0`.
3. Se pone `IsTrafficLightActive` en `false`.
4. Se pone `TrafficLightState` en `1.0`.
5. Se cambia `TrafficControlType` a `3`, indicando que ya no hay un control de parada activo.

---

## 12. Cambio de estado del semáforo: `SetLightState`

`SetLightState` recibe como entrada un nuevo estado del enum `E_TrafficLightState`.

Primero guarda ese estado en `CurrentState`. Después utiliza un `Switch` para aplicar los cambios visuales y lógicos según el color.

### Estado rojo

Cuando el estado es `red`:

1. El brillo del material rojo se pone en `50.0`.
2. El brillo del material amarillo se pone en `0.0`.
3. El brillo del material verde se pone en `0.0`.
4. `ColorState` se pone en `0.0`.
5. Se llama a `UpdateTrafficZone`.

### Estado verde

Cuando el estado es `green`:

1. El brillo del material rojo se pone en `0.0`.
2. El brillo del material amarillo se pone en `0.0`.
3. El brillo del material verde se pone en `50.0`.

4. `ColorState` se pone en `1.0`.
5. Se llama a `UpdateTrafficZone`.

## Estado ámbar

Cuando el estado es `amber`:

1. El brillo del material rojo se pone en `0.0`.
2. El brillo del material amarillo se pone en `50.0`.
3. El brillo del material verde se pone en `0.0`.
4. `ColorState` se pone en `0.5`.
5. Se llama a `UpdateTrafficZone`.

Al llamar a `UpdateTrafficZone` después de cada cambio, los coches que ya estaban dentro de la zona reciben inmediatamente el nuevo estado del semáforo.

---

## 13. Penalización por infracción de semáforo

La función `ApplyTrafficLightDeathPenalty` se ejecuta cuando un coche se salta un semáforo en rojo.

Recibe como entrada `TargetCar`, que es el coche infractor.

Primero calcula una penalización proporcional a la velocidad del coche:

```
DeltaTrafficLightDeathPenalty = Clamp(ForwardSpeed / MaxSpeed, 0, 1)^2 * MaxSpeed * ViolationPenaltyMultiplier
```

La penalización aumenta cuanto más rápido va el coche respecto a su velocidad máxima. Además, se multiplica por `ViolationPenaltyMultiplier`, que en los semáforos tiene un valor de `3.0`.

Después:

1. Se resta la penalización al `Fitness` del coche.
2. Se guarda el valor en `Debug_DeathPenalty`.
3. Se llama a `KillCar` con la razón `TRAFFIC_VIOLATION`.

---

## 14. Registro de muestras legales en semáforos

La función `AddLegalSample` recibe una velocidad llamada `SampleSpeed`.

Primero añade esa velocidad al array `LegalTrafficLightsSpeeds`.

Después comprueba si el array supera `MaxStoredSamples`. Si lo supera, elimina el elemento de índice `0`, es decir, la muestra más antigua.

Así, el semáforo mantiene una memoria limitada de las últimas velocidades legales observadas.

---

## 15. Umbral aprendido de velocidad en semáforo

La función `GetLearnedTrafficLightSpeedThreshold` devuelve un umbral de velocidad para decidir si el coche estaba suficientemente parado.

Si todavía no hay suficientes muestras legales, es decir, si `LegalTrafficLightsSpeeds` tiene menos de `MinSamplesForLearnedThreshold`, se devuelve un umbral inicial:

```
Threshold = MaxSpeed * 0.005
```

Este valor representa una velocidad muy baja, usada como referencia de parada.

Si ya existen suficientes muestras, la función calcula la media de todas las velocidades almacenadas en `LegalTrafficLightsSpeeds` y devuelve ese valor como umbral aprendido.

De esta forma, el sistema puede adaptarse a valores observados durante la simulación.

---

## BP\_Sign\_Stop

### 16. Función general

El `BP_Sign_Stop` controla el comportamiento asociado a una señal de STOP.

Cuando un coche entra en la zona del STOP, el blueprint le ordena detenerse en la línea correspondiente. Después comprueba si realmente se detiene, si lo hace antes de cruzar la línea y si permanece parado el tiempo mínimo necesario.

Cuando el coche ha realizado correctamente la parada, el sistema comprueba si las zonas de cruce están libres. Solo entonces libera al vehículo para continuar.

Si el coche abandona la zona sin haber realizado correctamente el STOP, se le aplica una penalización y muere con la razón `STOP_VIOLATION`.

---



## 17. Componentes principales

`SensorZone` es la caja de colisión que cubre la zona de detección del STOP.

`StopLine` es una flecha situada justo al final del STOP. Su posición se usa como línea objetivo de detención.

---

## 18. Variables principales

`CarsAlivedInCrossingZones` cuenta cuántos coches vivos hay dentro de las zonas de cruce asociadas.

`ViolationPenaltyMultiplier` multiplica la penalización por saltarse el STOP. Su valor es `2.0`.

`CrossingZones` es un array editable que contiene las zonas de cruce que deben estar libres antes de permitir que el coche continúe.

`MinSamplesForLearnedThreshold` indica cuántas muestras legales son necesarias para usar umbrales aprendidos. Su valor es `10`.

`MaxStoredSamples` limita el número máximo de muestras almacenadas. Su valor es `100`.

`LearnedThreshold` guarda el umbral de velocidad aprendido para considerar que el coche está parado.

`LearnedMinTime` guarda el tiempo mínimo aprendido que el coche debe permanecer detenido.

`LegalStopsSamples` almacena muestras legales de STOP. Cada muestra contiene velocidad y tiempo de parada.

`PendingWaitCars` almacena coches pendientes de volver a comprobar en `WaitForFullStop`.

`bWaitTimerActive` evita crear varios temporizadores simultáneos para la misma cola de espera.

`PendingCrossCheckCar` y `PendingCrossCheckRequestID` guardan el coche y el identificador de comprobación pendientes para revisar si el cruce está libre.

`LocalQueue` se usa como copia temporal de `PendingWaitCars` durante las recomprobaciones.

---

## 19. Entrada en la zona de STOP

Cuando un coche entra en `SensorZone`, se convierte el actor a `BP_VehicleAI2` y se obtiene su `BP_AiController`.

Después se configura el controlador del coche:

1. `TrafficControlType` se pone en `1`, indicando que el control actual es un STOP.
2. `StopTargetLocation` se establece con la posición mundial de `StopLine`.
3. `ForceStop` se pone en `true`.
4. `StoppedCorrectly` se pone en `false`.
5. `ApproachSpeed` del coche se guarda como la velocidad frontal actual.
6. `SensorEntryDistance` se calcula como la distancia entre la línea de STOP y el coche.
7. `CurrentStopWaitTime` se reinicia a `0.0`.

Por último, se llama a `WaitForFullStop` con ese coche como objetivo.

---

## 20. Salida de la zona de STOP

Cuando un coche sale de `SensorZone`, el sistema comprueba si había realizado correctamente el STOP.

Si `StoppedCorrectly` es `true`, se considera que el coche cumplió la norma. En ese caso:

1. `ApproachSpeed` se reinicia a `0.0`.
2. `ForceStop` se pone en `false`.
3. El coche se elimina de `PendingWaitCars`.

Si `StoppedCorrectly` es `false`, significa que el coche abandonó la zona sin completar correctamente la parada. En ese caso se llama a `ApplyStopDeathPenalty`.

---

## 21. Validación de parada completa: WaitForFullStop

`WaitForFullStop` comprueba de forma repetida si un coche ha realizado correctamente el STOP.

Primero valida que `TargetCar` exista. Si es válido, calcula dos umbrales:

- `LearnedThreshold`, obtenido con `GetLearnedStopSpeedThreshold`.
- `LearnedMinTime`, obtenido con `GetLearnedStopMinTime`.

Después incrementa `StopValidationTime` del coche en `0.1`.

A continuación comprueba si la velocidad absoluta del coche está por debajo del umbral aprendido:

```
abs(ForwardSpeed) < LearnedThreshold
```

Si el coche no está suficientemente parado, `CurrentStopWaitTime` se reinicia a `0.0`.

Si sí está parado, `CurrentStopWaitTime` aumenta en `0.1`.

---

## Condición principal de STOP correcto

El STOP se considera correcto cuando se cumplen varias condiciones a la vez:

1. La velocidad absoluta del coche es menor que `LearnedThreshold`.
2. El coche no ha sobrepasado la línea de parada.
3. La relación entre distancia restante y distancia de entrada es coherente con la velocidad actual respecto a la velocidad de aproximación.
4. `CurrentStopWaitTime` es mayor o igual que `LearnedMinTime`.

La comprobación de la línea se realiza usando un producto escalar entre la posición del coche, la posición de `StopTargetLocation` y el vector frontal de `StopLine`.

La idea general es comprobar que el coche se ha detenido antes de cruzar la línea, que está suficientemente cerca del punto de parada y que ha esperado el tiempo mínimo requerido.

---

## Caso en el que el coche se detiene tarde o mal

Si la condición principal de STOP correcto no se cumple, se revisa una segunda condición de error.

Esta condición detecta si el coche lleva demasiado tiempo de validación, está parado, pero se encuentra en una posición que indica que no se ha detenido donde debía.

El tiempo máximo de validación se compara con:

$$\text{LearnedMinTime} + \text{SensorEntryDistance} / \text{ApproachSpeed}$$

Si ese margen se supera y la posición relativa respecto a la línea no es válida, se considera una infracción de STOP y se llama a `ApplyStopDeathPenalty`.

---

## Reintento de comprobación

Si el coche todavía no ha cumplido correctamente el STOP, pero tampoco ha cometido una infracción definitiva, se añade a `PendingWaitCars`.

Después se comprueba si no hay ya un temporizador activo. Si no lo hay, se activa `bWaitTimerActive` y se crea un temporizador de `0.1` segundos que llamará a `Recheck_WaitForStop`.

De esta forma, el sistema no bloquea el flujo esperando al coche, sino que lo revisa periódicamente.

---

## Confirmación final del STOP

Cuando la condición principal de STOP correcto se cumple, el sistema espera `3.0` segundos y vuelve a comprobar la misma condición.

Esta segunda comprobación evita validar un STOP por un instante demasiado breve.

Si después de la espera el coche sigue cumpliendo la condición:

1. Se añade una muestra legal mediante `AddLegalSample`, guardando velocidad y tiempo de parada.
2. `StoppedCorrectly` se pone en `true`.
3. Se incrementa `StopCrossingCheckID` del coche.
4. Se llama a `CheckCrossingClear` con el coche y ese identificador.

---

## 22. Comprobación de cruce libre: CheckCrossingClear

Una vez que un coche ha realizado correctamente el STOP, no se le permite avanzar inmediatamente. Antes se comprueba si el cruce está libre.

La función recibe dos entradas:

- `TargetCar`, el coche que quiere cruzar.
- `RequestID`, el identificador de esta comprobación.

Primero se comprueba que `TargetCar` sea válido. Después se revisa que `RequestID` coincida con el `StopCrossingCheckID` actual del coche. Si no coincide, la comprobación se descarta porque pertenece a una petición antigua.

Si la petición es válida, se reinicia `CarsAlivedInCrossingZones` a `0`.

Después se recorren todas las `CrossingZones`. Para cada zona válida, se obtienen los actores `BP_VehicleAI2` que están dentro. Cada coche vivo encontrado, siempre que no sea el propio `TargetCar`, incrementa `CarsAlivedInCrossingZones`.

Cuando termina el recuento, se comprueba si hay coches vivos en las zonas de cruce.

### Si el cruce está ocupado

Si `CarsAlivedInCrossingZones > 0`, el sistema guarda `TargetCar` y `RequestID` como pendientes y crea un temporizador de `0.5` segundos para llamar a `Recheck_CrossingClear`.

Esto hace que el coche siga esperando hasta que el cruce quede libre.

## Si el cruce está libre

Si no hay coches vivos en las zonas de cruce, se vuelve a comprobar que el coche sea válido y que el `RequestID` siga siendo correcto.

Si todo es válido:

1. `TrafficControlType` se pone en `3`, indicando que ya no hay control de parada activo.
2. `ApproachSpeed` se reinicia a `0.0`.
3. `ForceStop` se pone en `false`.

En este momento el coche queda liberado y puede continuar la marcha.

---

## 23. Recheck\_WaitForStop

`Recheck_WaitForStop` se usa para revisar de nuevo los coches pendientes de completar el STOP.

Primero pone `bWaitTimerActive` en `false`.

Después copia `PendingWaitCars` en `LocalQueue` y limpia `PendingWaitCars`.

Finalmente recorre `LocalQueue` y llama a `WaitForFullStop` para cada coche.

Este patrón evita modificar directamente el array de pendientes mientras se está recorriendo.

---

## 24. Recheck\_CrossingClear

`Recheck_CrossingClear` vuelve a llamar a `CheckCrossingClear` usando `PendingCrossCheckCar` y `PendingCrossCheckRequestID`.

Se utiliza cuando el cruce estaba ocupado y se necesita volver a comprobarlo más tarde.

---

## 25. Penalización por infracción de STOP

`ApplyStopDeathPenalty` recibe como entrada `TargetCar`.

Primero calcula:

```
DeltaStopDeathPenalty = Clamp(ForwardSpeed / MaxSpeed, 0, 1)^2 * MaxSpeed *  
ViolationPenaltyMultiplier
```

En el STOP, `ViolationPenaltyMultiplier` vale `2.0`.

Después:

1. Se resta la penalización al fitness del coche.
  2. Se guarda en `Debug_DeathPenalty`.
  3. Se llama a `KillCar` con la razón `STOP_VIOLATION`.
- 

## 26. Registro de muestras legales de STOP

`AddLegalSample` recibe dos valores:

- `SampleSpeed`, velocidad legal observada.
- `SampleTime`, tiempo de parada legal observado.

Con ambos valores crea un `S_StopSample` y lo añade a `LegalStopsSamples`.

Si el número de muestras supera `MaxStoredSamples`, se elimina la muestra más antigua.

Así, el sistema conserva una memoria limitada de paradas correctas recientes.

---

## 27. Umbral aprendido de velocidad en STOP

`GetLearnedStopSpeedThreshold` devuelve el umbral de velocidad que se usa para decidir si un coche está parado.

Si todavía no hay suficientes muestras legales, devuelve:

```
Threshold = MaxSpeed * 0.005
```

Si ya existen al menos `MinSamplesForLearnedThreshold` muestras, calcula la media de las velocidades almacenadas en `LegalStopsSamples` y la devuelve como umbral aprendido.

---

## 28. Tiempo mínimo aprendido de STOP

`GetLearnedStopMinTime` devuelve el tiempo mínimo que un coche debe permanecer parado para validar el STOP.

Si todavía no hay suficientes muestras legales, devuelve:

```
MinTime = 3.0
```

Si ya hay suficientes muestras, calcula la media de los tiempos guardados en `LegalStopsSamples` y devuelve ese valor.

---

## BP\_Sign\_Yield

### 29. Función general

El `BP_Sign_Yield` controla la lógica de una señal de ceda el paso.

A diferencia del STOP, en un YIELD no siempre es obligatorio detenerse completamente. El coche debe reducir la velocidad lo suficiente y solo continuar cuando la zona de cruce esté libre.

Por eso, este blueprint no valida una parada completa con tiempo mínimo, sino que monitoriza si la velocidad del coche es adecuada en relación con su distancia a la línea de ceda y después comprueba si el cruce está libre.

Si el coche sale de la zona sin haber cumplido la validación, se le penaliza y muere con la razón `YIELD_VIOLATION`.

---

### 30. Componentes principales

`SensorZone` es la caja de colisión que cubre la zona de detección del ceda.

`YieldLine` es una flecha situada justo al final del ceda. Su posición se usa como referencia para comprobar la aproximación del coche.

---

### 31. Variables principales

`CarsAlivedCrossingZone` cuenta cuántos coches vivos hay en las zonas de cruce asociadas.

`ViolationPenaltyMultiplier` multiplica la penalización por infracción de YIELD. Su valor es `1.5`.

`CrossingZones` contiene las zonas que deben estar libres antes de permitir al coche avanzar.

`PendingMonitorCars` almacena coches pendientes de volver a revisar con `MonitorSpeed`.

`PendingYieldCheckCar` y `PendingYieldCheckRequestID` guardan el coche y el identificador de comprobación pendientes cuando el cruce no está libre.

`bMonitorTimerActive` evita crear varios temporizadores simultáneos para monitorizar coches pendientes.

`LocalMonitorQueue` se usa como copia temporal de la cola de coches pendientes.

---

## 32. Entrada en la zona de YIELD

Cuando un coche entra en `SensorZone`, se convierte el actor a `BP_VehicleAI2` y se obtiene su `BP_AiController`.

Después se configura el controlador:

1. `TrafficControlType` se pone en `2`, indicando que el control actual es un YIELD.
2. `StopTargetLocation` se establece con la posición mundial de `YieldLine`.
3. `ForceStop` se pone en `true`.
4. `StoppedCorrectly` se pone en `false`.
5. `SensorEntryDistance` se calcula como la distancia entre la línea de ceda y el coche.
6. `ApproachSpeed` se guarda como la velocidad frontal actual del coche.

Por último, se llama a `MonitorSpeed` con el coche como objetivo.

---

## 33. Salida de la zona de YIELD

Cuando un coche sale de `SensorZone`, el sistema comprueba si `StoppedCorrectly` está en `true`.

Si es `true`, el coche ha cumplido la validación del ceda. En ese caso:

1. `TrafficControlType` se pone en `3`.
2. `ApproachSpeed` se reinicia a `0.0`.
3. `ForceStop` se pone en `false`.
4. El coche se elimina de `PendingMonitorCars`.

Si `StoppedCorrectly` es `false`, se considera que el coche ha abandonado la zona sin respetar el ceda y se llama a `ApplyYieldDeathPenalty`.

---

## 34. Monitorización de velocidad: MonitorSpeed

`MonitorSpeed` comprueba si el coche se está aproximando al ceda de forma segura.

Primero valida que `TargetCar` exista. Si es válido, revisa una condición de salida:

- El coche ya está muerto.



- `ForceStop` ya no está activo.
- `StoppedCorrectly` ya es `true`.
- `TrafficControlType` ya no es `2`.

Si alguna de esas condiciones se cumple, no continúa.

Si el coche sigue pendiente de validación, incrementa `YieldValidationTime` en `0.1`.

Después compara dos valores normalizados:

1. La distancia relativa del coche hasta la línea de ceda.
2. La velocidad actual normalizada respecto a `MaxSpeed`.

La condición principal es:

```
DistanciaRelativaALaLinea >= VelocidadNormalizada
```

Esto significa que el coche se considera válido si su velocidad es suficientemente baja para la distancia que le queda hasta la línea. Si todavía está lejos, puede ir más rápido; si está cerca, debe ir más despacio.

### Si la condición se cumple

Si la relación entre distancia y velocidad es segura:

1. `YieldValidationTime` se reinicia a `0.0`.
2. `StoppedCorrectly` se pone en `true`.
3. Se incrementa `YieldCrossingCheckID` del coche.
4. Se llama a `CheckCrossing`.

### Si la condición no se cumple

Si el coche todavía no cumple la condición de velocidad segura, se añade a `PendingMonitorCars`.

Después, si no hay temporizador activo, se activa `bMonitorTimerActive` y se crea un temporizador de `0.1` segundos para llamar a `Recheck_MonitorStop`.

## 35. Comprobación de cruce en YIELD: CheckCrossing

Después de validar la aproximación al ceda, el sistema comprueba si el cruce está libre.

`CheckCrossing` recibe:

- `TargetCar`, el coche que quiere cruzar.
- `RequestID`, el identificador de la comprobación.

Primero valida que el coche exista. Después comprueba que `RequestID` coincida con `YieldCrossingCheckID` del coche. Si no coincide, se descarta porque es una comprobación antigua.

También comprueba si el coche está muerto o si ya no está bajo un control de tipo YIELD. En ese caso, no hace nada.

Si la comprobación sigue siendo válida, se reinicia el contador de coches vivos en la zona de cruce.

Luego se recorren todas las `CrossingZones`. Para cada zona válida, se obtienen los coches que están dentro. Cada coche vivo encontrado, excepto el propio `TargetCar`, incrementa el contador.

### Si el cruce está ocupado

Si hay coches vivos dentro de las zonas de cruce, el sistema guarda el coche y el `RequestID` como pendientes y crea un temporizador de `0.5` segundos para llamar a `Recheck_Crossing`.

### Si el cruce está libre

Si no hay coches vivos en las zonas de cruce, el sistema libera al coche:

1. `TrafficControlType` se pone en `3`.
2. `ApproachSpeed` se reinicia a `0.0`.
3. `ForceStop` se pone en `false`.

A partir de ese momento el coche puede continuar.

---

## 36. Recheck\_MonitorStop

`Recheck_MonitorStop` revisa de nuevo los coches que estaban pendientes de cumplir la condición de velocidad del ceda.

Primero pone `bMonitorTimerActive` en `false`.

Después copia `PendingMonitorCars` en `LocalMonitorQueue` y limpia `PendingMonitorCars`.

Finalmente recorre `LocalMonitorQueue` y llama a `MonitorSpeed` para cada coche.

---

## 37. Recheck\_Crossing

`Recheck_Crossing` vuelve a llamar a `CheckCrossing` usando `PendingYieldCheckCar` y `PendingYieldCheckRequestID`.

Se utiliza cuando el coche ya había cumplido la condición de velocidad, pero el cruce todavía estaba ocupado.

---

## 38. Penalización por infracción de YIELD

`ApplyYieldDeathPenalty` recibe como entrada `TargetCar`.

Primero calcula:

```
DeltaYieldDeathPenalty = Clamp(ForwardSpeed / MaxSpeed, 0, 1)^2 * MaxSpeed * ViolationPenaltyMultiplier
```

En el YIELD, `ViolationPenaltyMultiplier` vale `1.5`.

Después:

1. Se resta la penalización al fitness del coche.
  2. Se guarda el valor en `Debug_DeathPenalty`.
  3. Se llama a `KillCar` con la razón `YIELD_VIOLATION`.
- 

## BP\_CrossingZone

### 39. Función general

El `BP_CrossingZone` define una zona del mapa que debe estar libre antes de que un coche pueda continuar desde un STOP o un YIELD.

Su componente principal es `Zone`, una `BoxCollision` que cubre el área de cruce.

Los blueprints `BP_Sign_Stop` y `BP_Sign_Yield` consultan estas zonas para saber si hay otros coches vivos dentro. Si la zona está ocupada, el coche que espera en el STOP o YIELD no es liberado todavía. Si la zona está libre, se desactiva `ForceStop` y el coche puede continuar.

---

## S\_StopSample

### 40. Función general

`S_StopSample` es una estructura usada por `BP_Sign_Stop` para almacenar ejemplos de paradas legales.

Contiene dos valores:

`Speed`, que guarda la velocidad observada durante una parada válida.

`Time`, que guarda el tiempo durante el cual el coche permaneció correctamente detenido.

Estas muestras permiten calcular umbrales aprendidos de velocidad mínima y tiempo mínimo de parada.

---

## Resumen general del flujo

### 41. Funcionamiento conjunto del sistema

El sistema de intersecciones funciona conectando controles de tráfico con los coches mediante el `BP_AiController`.

Los semáforos son coordinados por `BP_IntersectionController`, que alterna fases verde, ámbar y rojo entre las direcciones norte-sur y este-oeste. Cada `BP_TrafficLight` actualiza visualmente su color y comunica a los coches si el semáforo está activo y cuál es su estado.

Cuando un coche entra en la zona de un semáforo, recibe una línea de parada, un estado de color y una distancia de entrada. Si sale de la zona con el semáforo en rojo y con una velocidad superior al umbral permitido, se considera una infracción y muere por `TRAFFIC_VIOLATION`.

Cuando un coche entra en una zona de STOP, se le obliga a detenerse en la línea de parada. El sistema comprueba que su velocidad sea suficientemente baja, que no haya sobrepasado la línea y que espere el tiempo mínimo necesario. Después revisa que el cruce esté libre antes de permitirle continuar. Si abandona la zona sin cumplir estas condiciones, muere por `STOP_VIOLATION`.

Cuando un coche entra en una zona de YIELD, no se exige una parada completa. En su lugar, se comprueba que la velocidad sea adecuada para la distancia restante hasta la línea de ceda. Si la aproximación es segura y el cruce está libre, el coche puede continuar. Si sale sin cumplir la validación, muere por `YIELD_VIOLATION`.

Las `BP_CrossingZone` permiten que STOP y YIELD tengan en cuenta el tráfico cruzado. Si otro coche vivo está dentro del cruce, el vehículo que espera no se libera todavía.

En conjunto, este sistema permite que los coches inteligentes interactúen con normas de tráfico básicas, aprendan umbrales a partir de ejemplos legales y reciban penalizaciones claras cuando incumplen una señal.